Healthcare MLOps

Acuity Bundle Prediction – Project Report

End-to-end machine learning system that recommends the right care bundle for a hospital patient

20,160	5	64	0.771	0.771
admissions	care-bundle classes	model features	test accuracy	test weighted F1

Executive Summary

Hospitals have limited ICU beds and staff, and deciding which level of care a patient needs is still largely manual. For conditions such as sepsis, every hour of delay increases mortality. This project builds a **clinical decision-support system** that predicts which of five **care bundles** a patient needs – **Routine Care, Enhanced Monitoring, Respiratory Support, Sepsis 1-Hour, Critical Care** – from aggregated vital signs and clinical scores (NEWS2, qSOFA, SIRS).

It covers the full ML lifecycle: data cleaning, exploratory analysis, a leakage-safe preprocessing pipeline, three tuned models tracked in **MLflow**, a **FastAPI** service with a React dashboard, **Docker** packaging, a **DVC** pipeline, weekly **Airflow** retraining and **GitHub Actions** CI. The selected model, **XGBoost**, reaches **77.1% accuracy and 0.771 weighted F1** on a time-based test set of the newest patients.

Contents

#	Section	#	Section
1	Project overview – problem, objective, users	6	Model selection – why XGBoost, why not others
2	Data overview	7	Evaluation
3	Exploratory data analysis (EDA)	8	Deployment & MLOps
4	ETL / preprocessing pipeline	9	Limitations & improvements
5	Tools & architecture	10	Interview quick reference

1. Project Overview

1.1 Problem

- ICU beds, nurses and rapid-response teams are scarce; triage decisions are manual and slow.
- Under-triage is dangerous: a missed sepsis or respiratory-failure patient deteriorates quickly.
- Over-triage wastes ICU capacity that another patient needs.

1.2 Solution

A **decision-support** model (the clinician always makes the final decision) that, for each patient, returns the recommended bundle **plus a probability for every class**, so staff can see how confident the model is and whether a second bundle is close behind. A batch endpoint scores a whole ER queue at once for capacity planning.

1.3 Objectives

- **ML objective:** classify patients into 5 bundles with high weighted F1 and good recall on the high-risk classes (Sepsis, Critical Care, Respiratory Support).
- **Engineering objective:** a production-style system that is **reproducible** (DVC + fixed seed), **tracked** (MLflow), **tested** (pytest + CI), **containerised** (Docker) and **retrainable** (Airflow).

1.4 Users

User	How they use the system
ER / triage nurses (primary)	Enter a new patient's vitals, get the recommended bundle instantly (`/predict`, dashboard)
Ward nurses	Spot deteriorating patients who need escalation (class probabilities + protocol text)
Doctors / rapid response team	Second opinion before escalating, e.g. starting the Sepsis 1-hour bundle sooner
Bed managers / operations	Score the whole queue to plan ICU beds and staffing (`/predict-batch`)
EHR systems	Call the REST API automatically – no manual data entry

1.5 The five target classes

Bundle	Clinical meaning	Share of data
Routine Care	Stable patient, standard observations	36.6%
Enhanced Monitoring	Some warning signs – more frequent observations	13.5%
Respiratory Support	Low SpO2 / high respiratory rate – oxygen or ventilation support	8.0%
Sepsis 1-Hour	Suspected sepsis – cultures, antibiotics, fluids within one hour	21.5%
Critical Care	Organ failure / very unstable – ICU-level care	20.4%

> *Patients are not users: the output is a clinical action plan that only trained staff can act on.*

2. Data Overview

Property	Value
File	`healthcare_data_acuity_bundle.xlsx` – sheet `ML_Features`
Size	20,160 rows × 52 columns (one row per admission)
Period	Admissions from 2024–2025 (used for the time-based split)
Type	**Synthetic** ICU-style dataset modelled on real clinical data such as MIMIC – not raw MIMIC
Target	`Acuity_Bundle` – 5 classes, imbalanced (largest 36.6%, smallest 8.0%)

2.1 Feature groups

Group	Columns	Used?
Identifiers & names	Patient_ID, patient / doctor names	No – privacy, no signal
Demographics	Age, Gender, Blood_Type, Insurance	Yes

Group	Columns	Used?
Vital-sign aggregates	Mean/Max/Std HR, Mean/Min SpO2, Mean/Min systolic BP, Max Temp, Mean/Max RR, Mean/Min GCS, Hypoxia & Tachycardia episodes, Vitals_Count	Yes
Clinical scores	NEWS2, qSOFA, SIRS, Risk_Score, Acuity_Level (Low–Critical)	Yes
Clinical context	Department, Diagnosis, Comorbidities, ICU_Admitted, ICU_Transfer	Yes
Outcome columns	LOS, Mortality, Discharge_Status, Readmission, Status	Yes – flagged as a leakage risk (Section 9)
Pre-encoded copies	*_Enc columns	No – duplicates of the raw columns
Bundle results	Compliance, Cost, etc.	No – only known after the bundle is given (leakage)

2.2 Why I describe it as synthetic

- Placeholder IDs and names (Patient_00000, Dr_11) and sequential Patient IDs.
- One pre-aggregated table, whereas real MIMIC comes as many linked tables (chartevents, admissions…).
- Deliberately injected errors – e.g. exactly 15 occurrences of each misspelling.
- Real MIMIC is de-identified, shifts dates into the 2100s and needs credentialed PhysioNet access with CITI human-research training.

2.3 Data-quality issues found

Issue	Evidence	Impact if ignored
Missing values	Max 2.5% (Mean_HR), SpO2 2.3%, BP 2.0%	Models fail on NaN; dropping rows loses ~18% of data
Duplicates	156 duplicate rows, 160 repeated Patient IDs	Bias + possible train/test leakage
Dirty categories	Gender in 8 spellings (M, m, Male, male…); ICU/icu/Icu; Emergency/er; Govt/PRIV/Self Pay	One real category split into many one-hot columns
Impossible values	Age −5 / 999, HR −20 / 450, SpO2 5 / 250, BP −30 / 550, Temp 5 / 120 °C, GCS −2 / 50	Distorted scaling and spurious splits
Class imbalance	Routine 36.6% vs Respiratory 8.0%	Model ignores the rare, high-risk classes

> **Say this:** *"I used a synthetic, MIMIC-style ICU dataset of about 20,000 admissions with aggregated vitals, clinical scores like NEWS2 and qSOFA, and a care-bundle label. Real data like MIMIC needs credentialed access, so synthetic data let me build and test the full pipeline. Because everything is config-driven, the same pipeline could be pointed at MIMIC-IV after aggregating its chart events."*

3. Exploratory Data Analysis (EDA)

Notebook `notebooks/01_eda.ipynb` · Tools: pandas, NumPy, Matplotlib, Seaborn. Goal: understand the data and turn every problem found into a concrete preprocessing decision.

#	Check	Method	Key finding
1	Shape & schema	`read_excel`, `shape`, `info()`	20,160 × 52; target = Acuity_Bundle
2	Column types	`select_dtypes()`	Vitals/scores numeric; Gender, Department, Insurance… text
3	Missing values	`isnull().sum()`, missing %	≤ 2.5% per column → impute, don't drop
4	Duplicates	`duplicated()`, repeated Patient_ID	156 duplicate rows, 160 repeated IDs
5	Category spelling	`value_counts(dropna=False)`	8 versions of Gender; mixed-case departments
6	Ranges	`describe()` min / max	Physically impossible vitals and ages
7	Quality table	Type, missing, % missing, unique per column	One-page plan for cleaning
8	Distributions	Histograms, box plots	Vitals are skewed with long tails

#	Check	Method	Key finding
9	Outliers	IQR rule (Q1 − 1.5·IQR, Q3 + 1.5·IQR)	e.g. Mean_HR 124, Mean_RR 151, BP 143 outliers
10	Target balance	`value_counts(normalize=True)`	Imbalanced: 36.6% down to 8.0%
11	Feature vs target	Box plots, count plots by bundle	Sicker bundles: higher HR, lower SpO2 and BP; Diagnosis & Department matter
12	Correlation	`corr()` + heatmap	Risk_Score ~ HR 0.46, SpO2 −0.45, RR 0.42; no extreme redundancy

3.1 From findings to actions

EDA finding	Preprocessing decision
Low missingness (≤ 2.5%)	Impute (median / most frequent) instead of dropping rows
Dirty category spellings	Normalise text and map synonyms before encoding
Physically impossible values	Clinical valid ranges → set the single bad value to NaN, then impute
Real but extreme vitals (IQR outliers)	**Keep them** – a HR of 160 is a very sick patient, exactly who we must detect
Skewed numeric distributions	Median imputation (robust to extremes) + StandardScaler
Class imbalance	Balanced class weights, stratified CV, weighted/macro F1 instead of accuracy
Admissions span 2024–2025	Time-based split to mimic real deployment

> **Say this:** *"The four key EDA findings were low missing values, so I could impute; dirty categories; physically impossible values; and class imbalance. Each finding became one preprocessing step."*

4. ETL / Preprocessing Pipeline

Code: `src/preprocess.py` + `src/feature_engineering.py` · all settings in `params.yaml` · run as the `preprocess` stage of the DVC pipeline.

EXTRACT Excel sheet (pandas) → **CLEAN** categories, ranges, flags → **SPLIT** by admission date 70 / 15 / 15 → **TRANSFORM** impute, scale, one-hot → **LOAD** CSVs + joblib artefacts

4.1 Extract

- `pd.read_excel` on the `ML_Features` sheet; path and sheet name come from `params.yaml`, never hard-coded.
- The raw file is a declared DVC dependency, so any change to it triggers a re-run of the pipeline.

4.2 Transform

#	Step	How	Why
1	Clean categories	Lower-case, strip spaces, map synonyms (male/m → M, govt → Government, emergency → ER, y/1/true → Yes)	Otherwise one gender becomes 8 one-hot columns
2	Fix impossible values	Clinical valid ranges in params.yaml (HR 20–250, SpO2 50–100, GCS 3–15, Temp 30–45 °C, Age 0–120…) – out-of-range → NaN	Typing errors; only that value is removed, the rest of the row is kept
3	Ordinal encoding	Acuity_Level: Low 1, Medium 2, High 3, Critical 4	Preserves natural order
4	Comorbidity flags	Comorbidities → Has_Hypertension, Has_Diabetes, Has_COPD, Has_CKD ("Both" sets two)	It is really a multi-label field
5	Feature selection	Keep only the feature list in params.yaml	Drops IDs (privacy), *_Enc duplicates, bundle-result columns (leakage)
6	Time-based split	Sort by Admission_Date: 70% train / 15% val / 15% test; undated rows → train	Train on the past, test on the future
7	Imputation	SimpleImputer – median (numeric), most frequent (categorical)	Median is robust to skew
8	Scaling	StandardScaler on numeric columns	Required for Logistic Regression

#	Step	How	Why
9	One-hot encoding	OneHotEncoder(handle_unknown="ignore")	An unseen category will not crash the API

4.3 Load

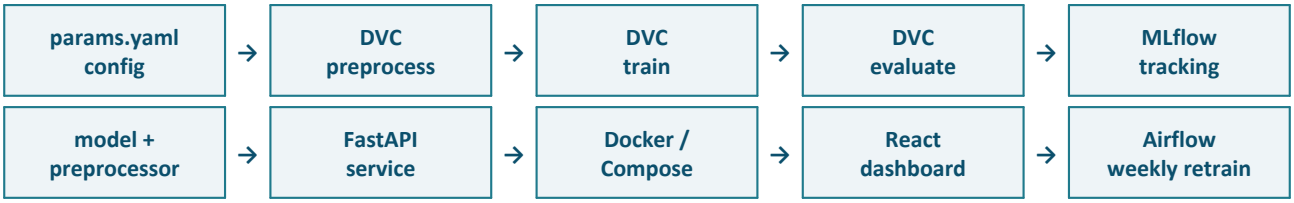
- `data/processed/train.csv`, `val.csv`, `test.csv` – DVC-tracked outputs.
- Steps 7–9 are one scikit-learn `ColumnTransformer`, **fit on train only** and saved as `models/preprocessor.joblib`; feature names saved to `feature_names.json`.
- The API loads the same preprocessor, so training and serving use the identical transformation.

Result: train 14,122 · validation 3,019 · test 3,019 rows · **64 features** after encoding.

Leakage controls: time-based split · preprocessor fit on train only · IDs and bundle-result columns removed · hyper-parameters chosen on CV/validation, test set used once.

5. Tools & Architecture

End-to-end flow



Layer	Tool	Why this tool	Alternative considered
Data / EDA	pandas, NumPy, Matplotlib, Seaborn	Standard, fast for 20k rows	Spark – unnecessary at this size
Preprocessing	scikit-learn Pipeline / ColumnTransformer	One serialisable object, fit on train only	Manual pandas code – leak-prone
Models	scikit-learn, XGBoost	Strong tabular performance, probabilities	LightGBM, CatBoost (next step)
Tuning	RandomizedSearchCV	Covers large grids cheaply (10 iter × 5 folds)	GridSearch (slow), Optuna
Experiment tracking	MLflow	Params, metrics, models per run; UI to compare	Weights & Biases (paid / SaaS)
Pipeline & data versioning	DVC	Reproducible stages; re-runs only what changed	Make / plain scripts
Configuration	params.yaml	All settings in one place, no hard-coding	Hydra
Serving	FastAPI + Pydantic	Input validation, auto Swagger docs, async, faster than Flask	Flask, BentoML
Front-end	React (Vite)	Interactive dashboard for nurses	Streamlit
Packaging	Docker, docker-compose	Same environment everywhere; API + MLflow together	Conda env
Orchestration	Apache Airflow	Scheduled weekly retraining with retries	Cron, Prefect
CI	GitHub Actions	Lint (flake8) → pytest → Docker build on every push	Jenkins
Testing	pytest	Unit tests for cleaning, integration tests for the API	–

6. Model Selection

6.1 Training approach

- Three candidate models, each tuned with **RandomizedSearchCV** – 10 iterations, **5-fold stratified CV**, scoring = **weighted F1**, `random_state=42`.
- Class imbalance: **class_weight="balanced"** for Logistic Regression and Random Forest; stratified folds for all.
- Every run (parameters, CV score, train/validation metrics, model artefact) is logged to MLflow as a nested run.
- The winner is picked on the **validation** set; the test set is touched only once, in the evaluate stage.

6.2 Search spaces

Model	Hyper-parameters searched
Logistic Regression	C {0.01, 0.1, 1, 10, 100}, solver {lbfgs, saga}, L2 penalty
Random Forest	n_estimators 100–500, max_depth {10, 20, 30, None}, min_samples_split/leaf, max_features {sqrt, log2}
XGBoost	n_estimators 100–500, max_depth {3, 5, 7, 10}, learning_rate 0.01–0.2, subsample, colsample_bytree, min_child_weight

6.3 Results

Model	Family / reason tried	Val accuracy	Val weighted F1
Logistic Regression	Linear baseline – simple, interpretable	0.743	0.750
Random Forest	Bagging – robust, non-linear	0.756	0.761
XGBoost (selected)	Boosting – state of the art on tabular data, regularised	**0.766**	**0.765**

Best XGBoost: 300 trees, max_depth 3, learning_rate 0.1, subsample 0.8, colsample_bytree 1.0, min_child_weight 3 (CV weighted F1 0.761).

6.4 Why XGBoost

- Highest validation F1 and accuracy of the three candidates.
- Captures non-linear effects and interactions (e.g. low SpO2 *and* high RR → respiratory support) that a linear model cannot.
- The tuned model is **compact** – 300 shallow trees of depth 3 – so it is fast to serve and less prone to overfitting than deep forests.
- Built-in regularisation, native handling of probabilities, and good tooling for explanation (SHAP).
- Test F1 (0.771) matched validation F1 (0.765), so it generalises to newer patients.

6.5 Why not other models

Model	Reason not chosen
K-Nearest Neighbours	Slow at prediction time (compares with every training row); weak with 64 dimensions
Support Vector Machine	Scales poorly with data size; no native probabilities (needs extra calibration)
Naive Bayes	Assumes independent features, but NEWS2 / qSOFA are calculated from the vitals
Deep learning (MLP)	Needs more data; harder to explain to clinicians; usually no better than boosting on 14k tabular rows
Single decision tree	Interpretable but high variance; ensembles dominate it
LightGBM / CatBoost	Good options – planned as next experiments, not a reason for rejection

7. Evaluation

7.1 Metrics and why

Metric	Why it is used
Weighted F1 (primary)	Balances precision and recall; weights each class by its size – the selection metric
Macro F1	Treats every class equally – exposes weak performance on the rare classes
Per-class recall	Clinically most important for Sepsis, Critical and Respiratory – a missed patient is worse than a false alarm
Confusion matrix	Shows *which* bundles are confused with each other
Accuracy	Easy to communicate, but misleading alone under class imbalance

7.2 Test-set results (newest 15% of patients, 3,019 rows)

0.771	0.771	0.716	0.776	0.771
accuracy	weighted F1	macro F1	weighted precision	weighted recall

Class	Precision	Recall	F1	Support
Routine Care	0.89	0.89	0.89	1,122
Enhanced Monitoring	0.54	0.65	0.59	371
Respiratory Support	0.70	0.50	0.58	246
Sepsis 1-Hour	0.72	0.71	0.71	685
Critical Care	0.80	0.80	0.80	595

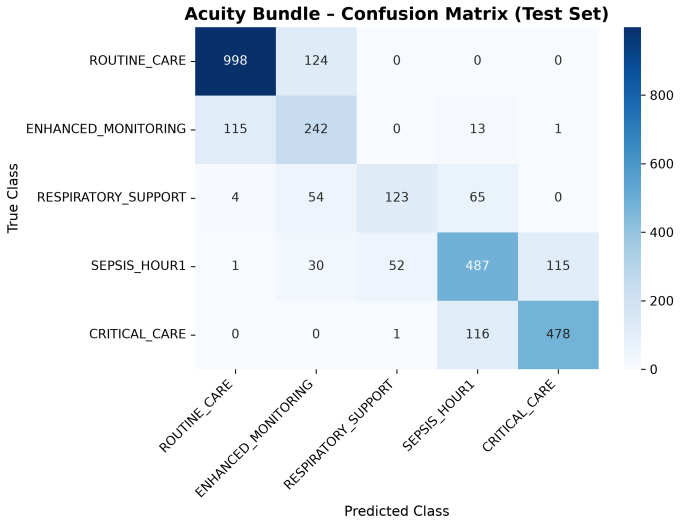


Figure 1 – Confusion matrix of XGBoost on the test set

7.3 Interpretation

- **Routine Care** (F1 0.89) and **Critical Care** (F1 0.80) are recognised well – the two ends of the acuity scale are the most distinct.
- **Respiratory Support** is the weakest class (recall 0.50): it is the rarest class (8%); of 246 such patients, 65 were predicted as Sepsis and 54 as Enhanced Monitoring.
- **Enhanced Monitoring** has low precision (0.54): 124 Routine Care patients were predicted as Enhanced Monitoring – borderline patients get pulled into it.
- **Sepsis vs Critical Care** is the main confusion between acute classes (115 and 116 patients each way) – both involve unstable vitals and low BP.
- Macro F1 (0.716) is lower than weighted F1 (0.771) – a direct signal that the minority classes need more work.

- **Generalisation:** train F1 0.838 → validation 0.765 → test 0.771. Validation and test agree, so performance is stable on future patients; the train–validation gap shows mild overfitting, which the shallow trees keep under control.

8. Deployment & MLOps

8.1 Prediction API (FastAPI)

Endpoint	Purpose
`GET /health`	Liveness check – is the service up and are the model artefacts loaded?
`GET /model-info`	Model name, parameters, features, classes and latest metrics
`POST /predict`	One patient → recommended bundle + probability for each of the 5 classes
`POST /predict-batch`	A list of patients (e.g. the whole ER queue) → predictions for all of them

Input is validated by a Pydantic schema; the API applies the same cleaning rules and the saved preprocessor before calling the model. Swagger documentation is generated automatically at /docs.

8.2 MLOps components

Component	What it does in this project
params.yaml	Single source of configuration: paths, features, valid ranges, split sizes, search spaces, MLflow settings
DVC pipeline	Three stages – preprocess → train → evaluate – with declared dependencies, outputs and metrics; `dvc repro` re-runs only the stages whose inputs changed
MLflow	Parent run per training pipeline with a nested run per model; logs params, CV score, train/val metrics and the model artefact. Falls back to a local SQLite store if the server is unavailable
Docker / Compose	python:3.11-slim image with the API, config and model; Compose runs the API and the MLflow server together
Airflow	DAG preprocess_data → train_models → evaluate_model, scheduled weekly with retries
GitHub Actions	On every push: install → flake8 lint → pytest → Docker image build
pytest	Unit tests for the cleaning functions and integration tests for the API endpoints

> **Say this:** *"Config lives in params.yaml; DVC runs preprocess, train and evaluate; MLflow tracks every experiment; the best model and preprocessor are served by FastAPI in Docker; CI tests every push; and Airflow retrains weekly. Everything is reproducible with a fixed seed and versioned config."*

9. Limitations & Improvements

9.1 Honest limitations

Limitation	Why it matters	Planned fix
Outcome columns in the feature list (LOS, Mortality, Discharge_Status, Readmission, Status)	Not known at triage time – inflates scores for early prediction	Remove and retrain; report the honest score
Duplicates not removed	156 duplicate rows can bias training and leak across splits	Drop duplicates before the split
Low recall on Respiratory Support (0.50)	Missed patients in a high-risk class	Class weights / sample weights for XGBoost, per-class threshold tuning
Cleaning logic partly duplicated in the API	Risk of training–serving skew	One shared cleaning module used by both
NEWS2, qSOFA, SIRS not yet API inputs	API relies on defaults/imputation for them	Add them to the Pydantic schema
Synthetic data	Real-world performance unknown	Validate on MIMIC-IV after aggregating chart events

9.2 Roadmap

- **MLflow Model Registry** with a promotion gate – a new model is promoted only if it beats the current one.

- **Drift monitoring** with Evidently (PSI / KS tests on live inputs) to trigger retraining.
- **SHAP explanations** per prediction so clinicians see *why* a bundle was recommended.
- **Probability calibration** so a 70% score really means 70%.
- Try **LightGBM / CatBoost**; multi-stage Docker build; cloud deployment with authentication and audit logs.

> **Don't claim as done:** model registry, drift monitoring, multi-stage Docker or production deployment. These are next steps.

10. Interview Quick Reference

10.1 30-second pitch

> *"I built an end-to-end MLOps system that predicts which care bundle a hospital patient needs – Routine, Enhanced Monitoring, Respiratory Support, Sepsis 1-Hour or Critical Care – from vitals and clinical scores like NEWS2 and qSOFA. I cleaned a messy 20k-row dataset, built a leakage-safe pipeline with a time-based split, tuned three models tracked in MLflow, and served the best one, XGBoost, through FastAPI and a React dashboard in Docker, with DVC, Airflow retraining and GitHub Actions CI. It reaches 77% accuracy and 0.77 weighted F1 on the newest patients."*

10.2 Likely questions

Question	Short answer
Why a time-based split?	It mimics real use – train on the past, predict the future. A random split leaks future patterns.
How did you handle imbalance?	Balanced class weights, stratified CV, and weighted / macro F1 instead of accuracy.
Precision or recall?	Recall for the dangerous classes – missing a sepsis patient is worse than a false alarm.
Why not remove IQR outliers?	A heart rate of 160 is a real, very sick patient. I removed only physically impossible values.
Why median imputation?	Vitals are skewed; the median is not pulled by extreme values.
Why fit the preprocessor on train only?	To avoid leaking validation/test statistics, and so the API uses the exact same transform.
Why XGBoost over Random Forest?	Slightly better F1, a much more compact model (300 depth-3 trees), and stable val → test scores.
How would you detect drift?	Compare live input distributions with training data (PSI / KS) and retrain when they drift.
Hardest part?	Messy data, and making sure nothing leaked from the future.
Is it safe to use clinically?	It is decision support only; it needs validation on real data, calibration and explanations first.

10.3 Key numbers

Item	Value	Item	Value
Rows × columns	20,160 × 52	Classes	5
Train / val / test	14,122 / 3,019 / 3,019	Features after encoding	64
Best model	XGBoost, 300 trees, depth 3	Validation F1	0.765
Test accuracy	0.771	Test weighted F1	0.771
Test macro F1	0.716	Best / weakest class	Routine F1 0.89 / Respiratory recall 0.50